# Lesson 5 Intelligent Stacking

## 1. Program Logic

When intelligent stacking game starts, the robotic arm can recognize two kinds of items and stack the items to different area by category. Color blocks and blocks with tags respectively represent one kind of items. The overall process consists of three parts, including recognizing, picking and stacking.

First, the robotic arm will recognize the object within the recognition area. It is divided into tags recognition and color recognition, and recognition methods of these two cases are different.

Color recognition is realized by Lab color space. Convert the RGB color space to Lab, image binarization, and then perform operations such as expansion and corrosion to obtain an outline containing only the target color. Use circles to frame the color outline to realize object color recognition.

For tag recognition, the robotic arm target the tag outline through locating, image segmentation and outline searching. Next, it comes to quadrilateral detection, and form a quadrilateral through connecting four corner points. Then, it will code and decode the target tag, and gets corresponding ID.

The next step is to determine whether the block continues to move or not. If it detect that the blocks stop moving for a while, it will begin picking the block. And there is differences between the ways to pick color block and to pick block with tag.

Block with tag: The robotic arm will determine the picking order through comparing the ID number it obtained before. Block with smaller ID number will be picked first.

Color block: after obtaining the recognition area of the corresponding colors, ArmPi FPV will compare the data it gain. It will make multiple judgments and takes the average to compare their sizes. Then it determines the picking order of the goods in the same color based on their area. Priority will be given to object with large area.

Lastly, stack the color blocks and blocks with tags in different stacking area. When stacking two kinds of block, the robotic arm will stack the block on the below block. Having completing the task, the robotic arm will return to original posture.

The source code is located in **/home/ubuntu/armpi_fpv/src/object_pallezting/scripts**.

```
# place position x, y, z(m)
place_position = {'red':   [-0.13, -0, 0.01],
                  'green':[-0.13, -0, 0.01],
                  'blue':  [-0.13, -0, 0.01],
                  'tag1':  [ 0.13, -0, 0.01],
                  'tag2':  [ 0.13, -0, 0.01],
                  'tag3':  [ 0.13, -0, 0.01]}

# get the max area contour
# The parameter is a list of contours to be compared
def getAreaMaxContour(contours):
    contour_area_temp = 0
    contour_area_max = 0
    area_max_contour = None

    for c in contours:  # detect all contours
        contour_area_temp = math.fabs(cv2.contourArea(c))  # Calculate the contour area
        if contour_area_temp > contour_area_max:
            contour_area_max = contour_area_temp
            if contour_area_temp > 100:  # Only when the area is larger than the setting, the contour of the largest area is effective to filter interference
                area_max_contour = c

    return area_max_contour, contour_area_max  # return to the
```

# 2. Operation Steps

The entered command must be strictly distinguished between uppercase and lowercase and spaces, and the keywords support the "TAB" key completing.

## 2.1 Preparation

① Please place the map on even desk and the robotic arm should be placed on the corresponding area on the map.

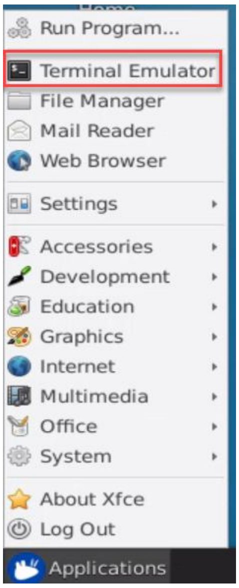
② Prepare 3 blocks in red, green and yellow and 3 blocks labeled with tag1, tag2 and tag3. And place them randomly on the recognition area of the map. Note: the interval between each blocks is not less than 3cm.

③ Turn on the robotic arm and wait to finish.

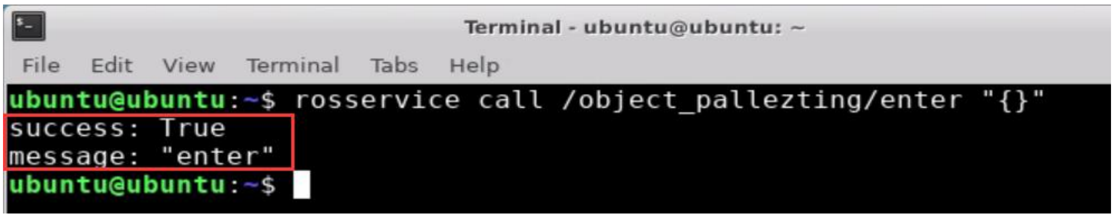
## 2.2 Enter the game

① Turn on ArmPi FPV and connect to Raspberry Pi desktop through NoMachine.

② Click "Applications" button and select " Terminal Emulator" in pop-up interface to open the command line terminal.



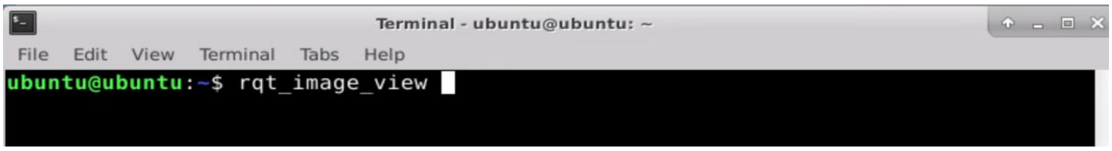
③ Enter “**rosservice call /object_tracking/enter "{}"**" command, then press “Enter” to start the game. After the game starts, a print prompt will appear, as shown in the figure below.



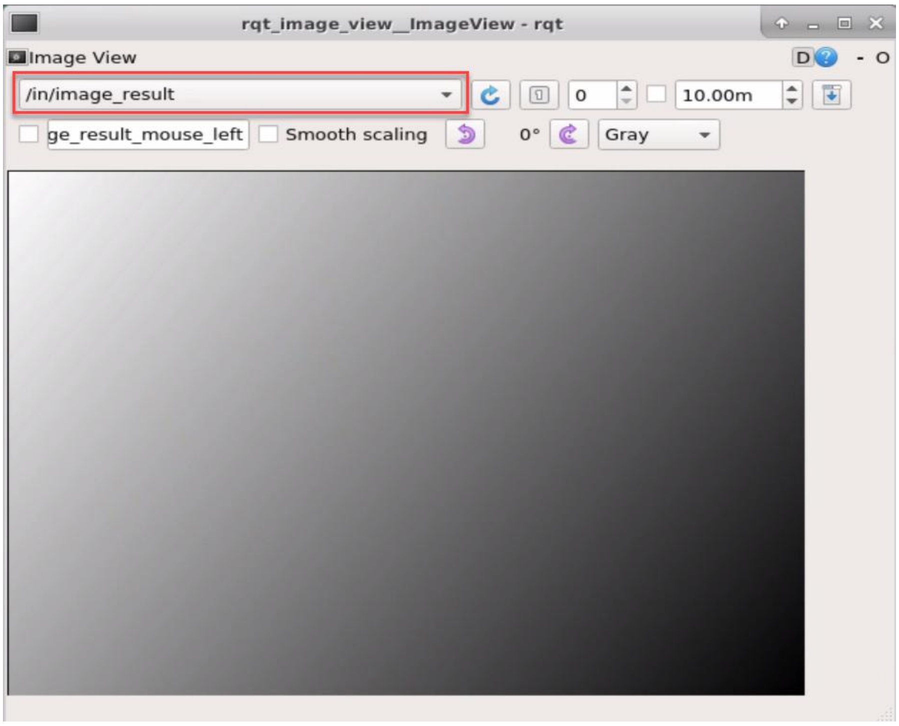

## 2.3 Turn On the Image

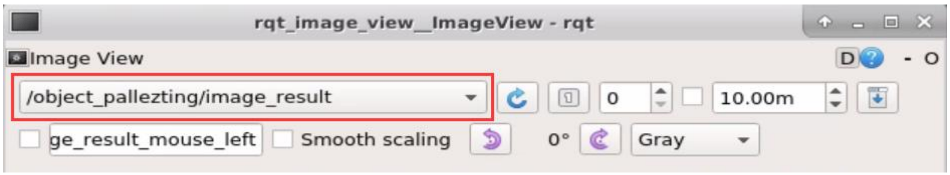
① When entering the game, we need to turn on image returned by camera with rqt. Please go back to the desktop and click the “Application” icon at bottom-left corner to open a new terminal.

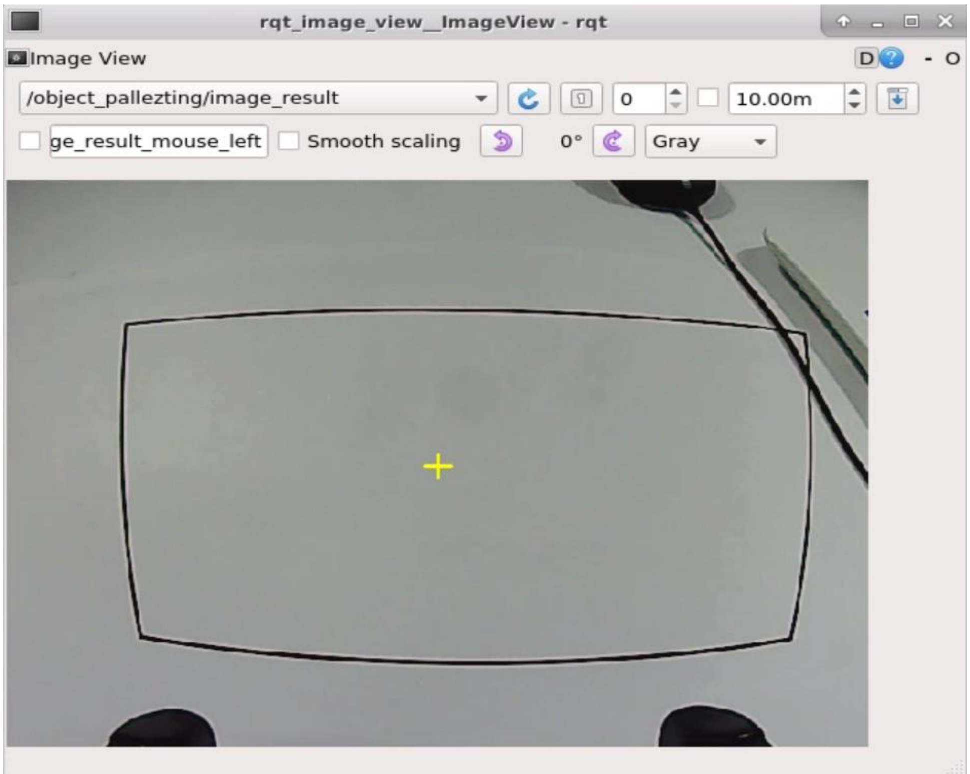
② Enter “**rqt_image_view**” command and press “Enter”. Then open rqt after few seconds.




③ As shown in the following figure, select “**/object_pallezting/image_result**” and other settings remain the same.


rqt_image_view__ImageView - rqt
Image View
/in/image_result
0
10.00m
ge_result_mouse_left
Smooth scaling
0°
Gray

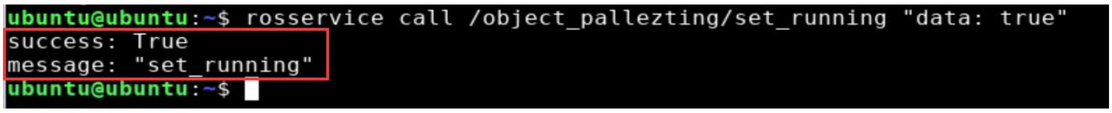

rqt_image_view__ImageView - rqt
Image View
/object_pallezting/image_result
0
10.00m
ge_result_mouse_left
Smooth scaling
0°
Gray


rqt_image_view__ImageView - rqt
Image View
/object_pallezting/image_result
0
10.00m
ge_result_mouse_left
Smooth scaling
0°
Gray

Note: after the image is turned on, please be sure to select the topic corresponding to the game you play. Otherwise, the recognition process can not be displayed normally if you start other games.

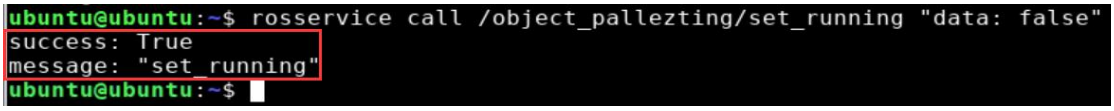
## 2.4 Start the Game

Please go back to the terminal opened in “2.2 Enter the Game” and enter **rosservice call /object_pallezting/set_running "data: true"** command. When the prompt, shown in the red box, appears, it means you start the game successfully.

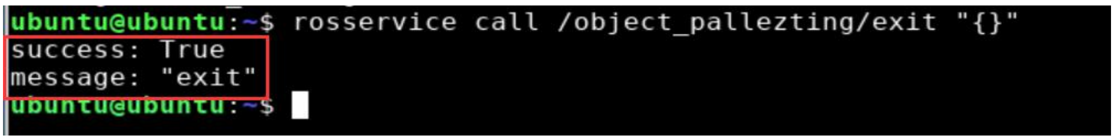



## 2.5 Pause and Quit the Game

You can enter “**rosservice call /object_pallezting/set_running "data: false"**" command to pause the game. Referring to “2.4 Start the game”, you can change the targeted color or label after a pause.




You can enter “**rosservice call /object_sorting/exit "{}"**” command to quit the game.




Note: the current game will continue as Raspberry Pi is powered on. To avoid occupying too much RAM, please quit the current game according to the instruction above before playing other AI vision game.

If you want to turn off the image returned by camera, you can turn back to open rqt terminal and press “Ctrl+C”.

## 3. Function Realization

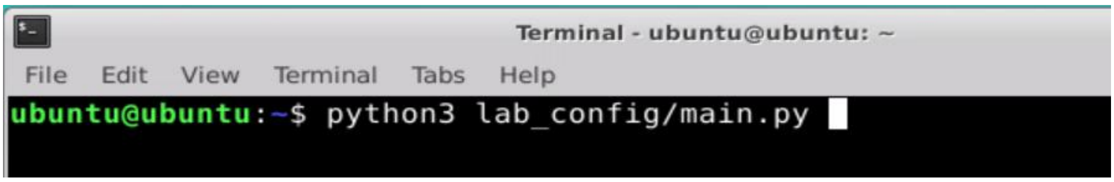
When the game starts, the robotic arm will pick blocks with tag and color block in turns, then it will stack them in the corresponding area.

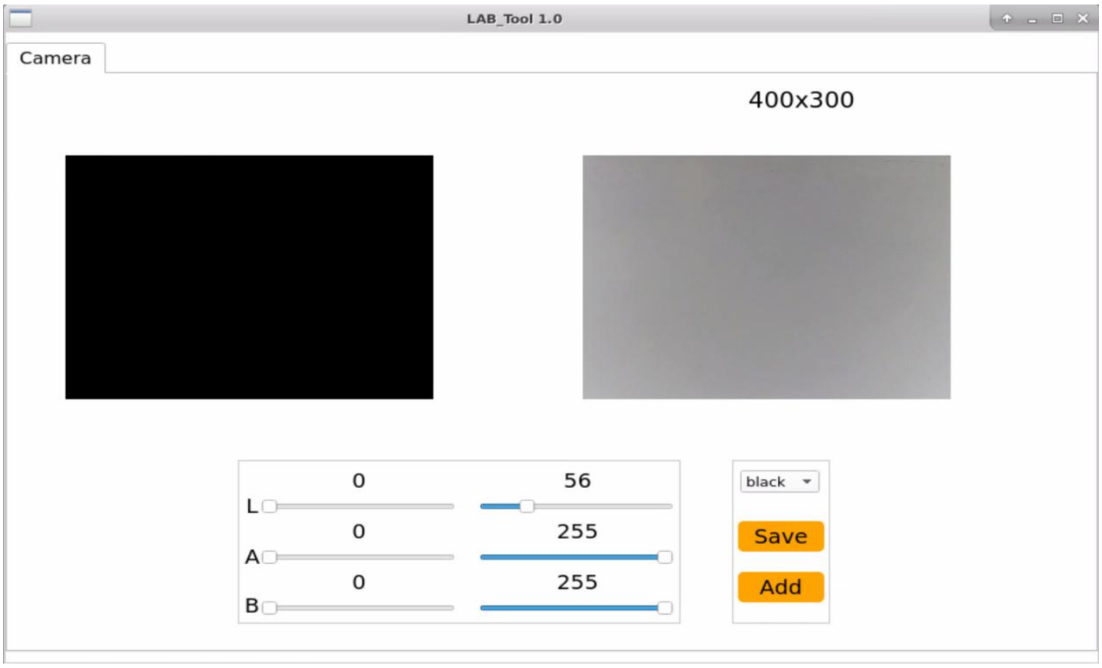
## 4. Function Extension

### 4.1 Add New Recognition Color

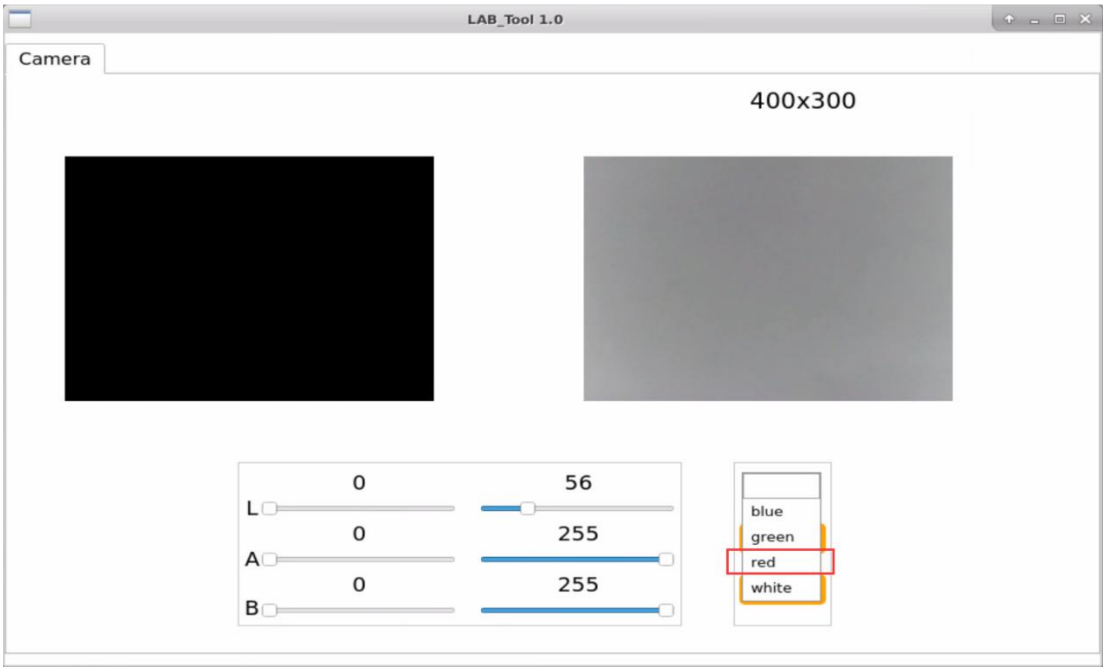
There are three built-in recognition colors in the color tracking program, including red, green and blue. Besides these three colors, you can add new recognition colors. Let’s take orange for example.

① Please open the terminal interface and enter “**python3 lab_config/main.py**” command to open color threshold debugging tool. If there is no image returned by camera in the pop-up interface, it means the camera has not been connected successfully. And you should go back to check the camera connection.



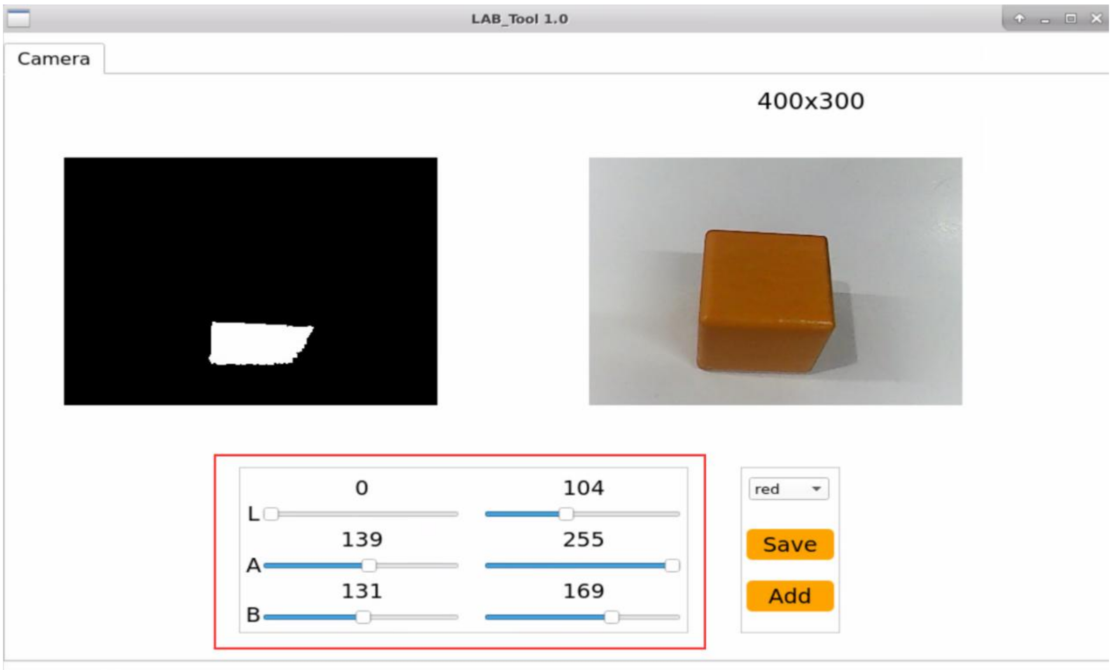


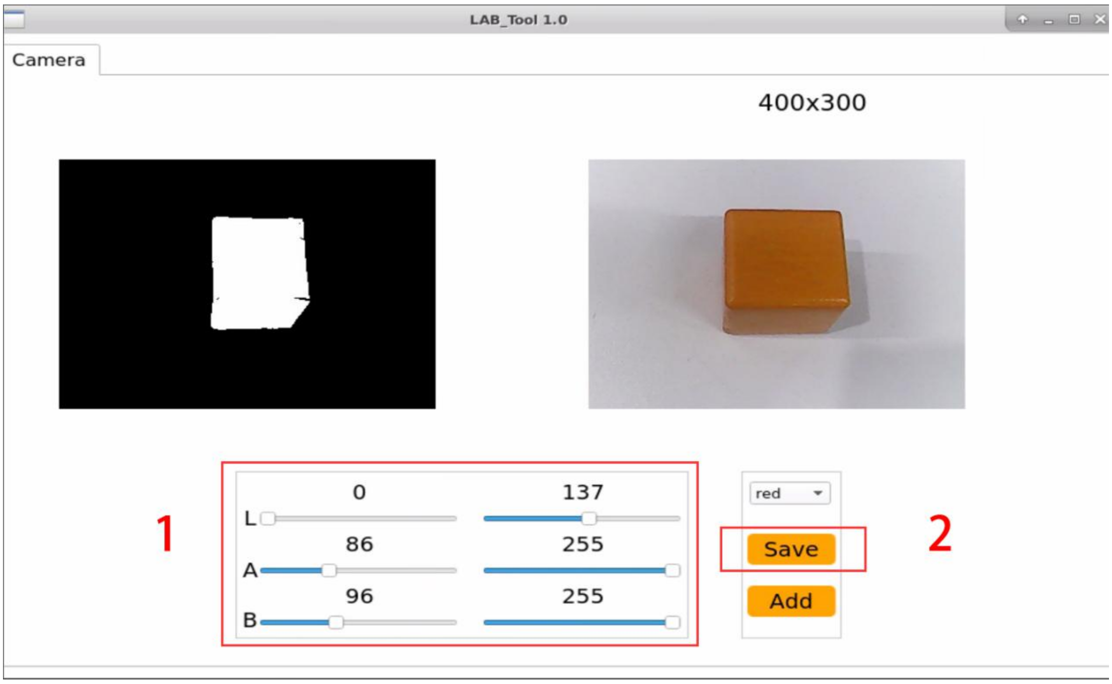

② After connecting successfully, please select "red".




③ At this time, the image returned by camera is on the right of the interface and the box to recognize color is on the left. Focus the camera on yellow item, then drag the six sliders below until the orange part of the left screen becomes white and area outside of orange become black, and then click " Save" button to keep the modified data.

It is recommended to use screenshot to record the initial value, which is beneficial for you to change the value back. Or you can follow step ③ to set red as the recognition color.




④ Please follow the first three steps in "2. Operation Steps", from 2.1 to 2.3, to play the intelligent stacking games.

In the previous steps, we have saved orange as red, therefore we should select the targeted color to “red” before starting the game.

⑤ Please place the orange item in front of the camera. Having recognized, the ArmPi FPV will pick it to color sorting area. If you want to modify to other colors, please operate as the above steps.